CodeHolic: Bitcask Key-Value Store

Competition: "Kill It and Prove It" 24-Hour Engineering Hackathon
GitHub Repository: https://github.com/IshwaryaNP/CodeHolic
Technology: Go 1.22.6 (Standard Library Only: `os`, `bufio`, `hash/crc32`, `sync`, `net/http`)
Core Problem: High-performance persistence with mathematical correctness under sudden termination (`kill -9`).

1. Executive Summary

CodeHolic is an embedded, append-only key-value database built from first principles in Go. The architecture guarantees that no acknowledged write is ever lost, no torn or uncommitted transaction is ever partially applied, and recovery from sudden process death completes in under 1 second.

> **Key Achievements:**
> - **Crash Safety Invariant:** 100% adherence across hundreds of unannounced `kill -9` interruptions.
> - **Ultra-Fast Recovery:** Rebuilds the in-memory RAM index for 1,000,000 keys in **~0.638s** (Limit: < 2.0s).
> - **Online Compaction:** Safely reclaims deleted and overwritten disk space with atomic cross-platform file replacement.
> - **Full Test Suite:** 30 automated tests passing with the Go Race Detector (`-race`) enabled.

2. On-Disk Binary Format & Storage Engine

The authoritative database state resides strictly within a single append-only `.db` file on disk. Each entry is serialized with a fixed 15-byte header followed by the variable-length key and value:

```
+---------------+----------------+---------------+---------------+---------------+----------------+
|  CRC32 (4B)   | TotalLen (4B)  |  OpType (1B)  |  KeyLen (2B)  |  ValLen (4B)  |  Key (NB)
+---------------+----------------+---------------+---------------+---------------+----------------+
0               4                8               9               11              15              1
```

- **CRC32 (IEEE) (Bytes 0..4):** Calculated incrementally over `header[4:15] + key + value` to guarantee zero allocation overhead on writes.
- **TotalLen (uint32) (Bytes 4..8):** Exact length of the whole record (`15 + KeyLen + ValLen`).
- **OpType (uint8) (Byte 8):** `1 = OpPut`, `2 = OpDelete` (tombstone with ValLen=0), `3 = OpBatchBegin`, `4 = OpBatchCommit`.
- **In-Memory RAM Index:** A thread-safe hash map (`sync.RWMutex`) mapping `key -> fileOffset` of the latest valid `OpPut` record.

3. Transactional Write Path & Compaction Ordering

3.1 Atomic Write Sequence

1. **Acquire Lock:** `DB.mu.Lock()` serializes concurrent writers and guards the per-DB write buffer `encBuf`.
2. **Encode Record:** Computes header fields and IEEE CRC32 into contiguous bytes.
3. **Append to File:** Issues a single `file.Write(encBuf)` at the current append offset.
4. **Durable Flush (fsync):** Calls `file.Sync()` (`FlushFileBuffers` on Windows) to force file blocks out of OS cache onto physical storage.
5. **Update RAM Index:** Updates `index.Set(key, offset)` only *after* physical sync succeeds.
6. **Emit ACK:** Sends acknowledgment to caller. If killed before ACK, the write was not acknowledged, satisfying the invariant.

3.2 Crash-Proof Compaction Sequence

1. Snapshot all live keys sorted by original file offset to maintain sequential read performance.
2. Stream live records into a temporary file (`<path>.tmp`) using a 1MB buffered writer.
3. Explicitly flush `tmpFile.Sync()` and close `tmpFile`.
4. Close the active `db.file` (mandatory on Windows to release file locks).
5. Perform atomic directory replacement via `os.Rename(tmpPath, dbPath)`.
6. Reopen the newly compacted database file and atomically swap the in-memory index map.

4. Hackathon Deliverables Overview

Deliverable	File Location	Description
Deliverable 1: Engine CLI	`cmd/kvdb/main.go`	Mandatory 8-command CLI wrapper (`open` , `put` , `get` , `del` , `batch` , `dump` , `check` , `serve`).
Deliverable 2: Torture Tool	`cmd/torture/main.go`	Chaos testing tool running concurrent workers, random SIGKILLs, independent oracle verification, and targeted modes.
Deliverable 3: Attack Analysis	`BREAKING.md`	12-row Crash Points Table, ordering proofs, design trade-offs, and failure post-mortems.
Deliverable 4: Prompt Audit Log	`PROMPTS.md`	Chronological audit log of all technical prompts, inquiries, and architecture discussions.
Deliverable 5: Robustness Proof	`ARCHITECTURE.md`	Mermaid architecture diagram, 32-row scenario verification table (Groups A–F), and formal guarantees.
Interactive Web UI	`cmd/ui/`	Zero-dependency local dashboard visualizer embedded via `go:embed` .

5. 15-Minute Judge Demo Walkthrough Guide

Minutes 0 to 1: Clean Clone & Build

```
go build -o kvdb.exe ./cmd/kvdb && go build -o torture.exe ./cmd/torture
```

Expected: Builds cleanly with standard library only in seconds.

Minutes 1 to 4: Torture Tool Run (Judge's Seed)

```
./torture --iterations 200 --seed 12345
```

Runs 2 concurrent writers against `kvdb serve` , fires unannounced `SIGKILL` s at random millisecond intervals, restarts with `open` , runs `check` and `dump` , and validates that every single ACKed write exists with zero torn values.

Minutes 4 to 6: Targeted Chaos Modes

```
./torture --iterations 100 --seed 7 --mode compaction
./torture --iterations 100 --seed 42 --mode recovery-crash
./torture --iterations 100 --seed 999 --mode large-values
```

Expected: `PASS` Zero invariant violations across all modes.

Minutes 6 to 8: Manual Interactive Kill Test

```
# Start long-running serve engine
./kvdb serve ./judge.db

# Feed batch operations from stdin, then kill process abruptly:
taskkill /F /IM kvdb.exe

# Verify integrity:
./kvdb check ./judge.db
# OUTPUT: OK
```

Minutes 8 to 12: Crash-Here Code Inspection Answers

- **Process dies during `file.Write()` before `Sync()`:**
  Disk state: Incomplete record bytes at tail (torn write).
  Recovery action: `DecodeHeader` detects length mismatch or invalid CRC32; rolls back file via `Truncate(validEnd)`.
- **Process dies in batch after 5 records, before `BatchCommit`:**
  Disk state: Valid `BatchBegin` and 5 records on disk; no commit marker.
  Recovery action: Recovery detects open batch state at EOF; rolls back entire batch to `batchBeginOffset`.
- **Process dies in compaction while streaming to `.tmp`:**
  Disk state: Authoritative `.db` is untouched. Partial `.db.tmp` file exists.
  Recovery action: Recovery detects and unlinks `.db.tmp` immediately on startup.

Minutes 12 to 15: Design Extension (Key Expiration / TTL)

Judge's Question: "*Add an expiry time (TTL) to every key. Keys past expiry must be invisible to reads and purged during compaction. What changes and what breaks?*"
Engineered Answer:

1. **Header Change:** Expand fixed header from 15B to 23B by adding an 8-byte `ExpiresAt` Unix timestamp. `TotalLen = 23 + KeyLen + ValLen`.
2. **Read Path:** `Get()` checks `hdr.ExpiresAt <= time.Now().UnixMilli()`; returns `ErrNotFound` if expired.
3. **Compaction:** Omit expired records from `.tmp`, reclaiming space.
4. **Subtle Traps Identified:** Wall-clock adjustments (NTP backward drift) can resurrect expired keys (requires monotonic clock); batch updates with mixed TTLs; and compaction timestamp drift (must freeze `compactionStartTimestamp`).

6. Verification & Test Metrics

- **Race Detector:** `go test -race ./...` → `ALL 30 TESTS PASS`.
- **Recovery Benchmark:** 1,000,000 keys loaded and indexed in **0.638s**.

- **Git Status:** Fully committed and pushed to GitHub main and feature branches.